Supplementary Material — CamLoop4D: Controllable 4D Scene Generation by Closing the Camera Loop between Generation and Reconstruction

H. Li[1] and J. Chen[2†]

[1]Independent Researcher, Chengdu, China
[2]Tsinghua University, Beijing, China

† Corresponding author: junhao-c24@mails.tsinghua.edu.cn

Appendix A: Overview

In this supplementary material, we provide:

- Dynamic Scene Representation details in Section B;
- Implementation Details, including the system-level generation/evaluation protocol and the camera-adherence (ATE) diagnostic, in Section C;
- Per-Scene Reconstruction Metrics and paired significance in Section D.2;
- Overfitting analysis (train vs. held-out) and the trainable-bases conditioning control in Section D.3;
- Unified Benchmark Evaluation, per-category results, and baseline reproduction in Section D.4;
- Additional 4D Generation results in Section D.5;
- 3D Tracking and Motion Analysis in Section D.6.

Appendix B: Dynamic Scene Representation

B.1. Motion Bases

We propose a novel persistent 3D Gaussian representation with a hybrid motion basis (see the system overview in the main paper), combining six fixed bases $\{G_{\text{tx}}, G_{\text{ty}}, G_{\text{tz}}, G_{\text{rx}}, G_{\text{ry}}, G_{\text{rz}}\}$ and $B-6$ trainable ones $\{G_7, \ldots, G_B\}$, where each Gaussian's motion is modelled as a linear combination of $\mathfrak{se}(3)$ Lie algebra elements followed by the exponential map, enabling fine-grained control of complex dynamics.

The six fixed bases are the standard generators of $\mathfrak{se}(3)$: three unit translation generators along the X, Y, and Z axes, and three infinitesimal rotation generators about these axes. These remain frozen during training to preserve global rigidity and structural priors. Each trainable basis is initialised to zero and parameterised as a learnable element of $\mathfrak{se}(3)$ (represented via a 3D rotation vector and a 3D translation vector in implementation), enabling adaptive, data-driven refinement of motion details.

The general form of an $\mathfrak{se}(3)$ generator is (Equation (1)):

$$G = \begin{bmatrix} [\omega]_\times & v \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}, \tag{1}$$

where $[\omega]_\times \in \mathbb{R}^{3\times 3}$ is the skew-symmetric matrix of the angular velocity $\omega \in \mathbb{R}^3$, and $v \in \mathbb{R}^3$ is the translational velocity. The six fixed generators are defined as follows (Equations (2) to (7)):

$$G_{\text{tx}} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \tag{2}$$

$$G_{\text{ty}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \tag{3}$$

$$G_{\text{tz}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \tag{4}$$

$$G_{\text{rx}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \tag{5}$$

$$G_{\text{ry}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \tag{6}$$

$$G_{\text{rz}} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{7}$$

Each trainable basis G_j ($j = 7, \ldots, B$) shares the same $\mathfrak{se}(3)$ structure as Equation (1), with $[\omega_j]_\times$ and v_j learned from data. In implementation, following standard practice [WYG*25, KLD24], these are compactly represented as 6D vectors (3D rotation vector + 3D translation) and converted to $\mathbb{SE}(3)$ elements via the exponential map during the forward pass.

Appendix C: Implementation Details

C.1. Dataset Details

To evaluate 3D tracking, we conduct experiments on iPhone Dataset [GLT*22], DAVIS Dataset [PPTM*16] and CamLoop4D-generated Dataset.

The iPhone Dataset contains 14 sequences of 200–500 frames each, capturing diverse, non-repetitive motions across multiple categories, such as generic objects, humans, and pets, in challenging real-world scenes.

The DAVIS dataset [PPTM*16] contains about 30 to 100 frames of real-world video covering multiple scenes and motion dynamics.

C.2. Training Details

We implement CamLoop4D with PyTorch. Our approach leverages the zero-shot capabilities of powerful pre-trained video diffusion models within a fully integrated pipeline. Specifically, CamLoop4D operates efficiently and requires minimal computational resources; all experiments run on a computer equipped with an Intel Core i9-12900K (3.50GHz) processor and a single NVIDIA GeForce RTX 3090 GPU.

C.3. Coordinate, Intrinsics, and Scale Conventions

The generator and reconstructor share a single geometric convention so that no separate scale or frame resolution is required. (i) *Intrinsics*: the reconstructor uses the same intrinsics $\mathbf{K}$ that condition the generator. (ii) *World frame*: we anchor the world frame at the canonical frame t_0; all per-frame transforms $T_{0:t}$ are expressed relative to it. (iii) *Scale*: the metric scale is fixed by the specified camera trajectory. Depth Anything [YKH*24] predictions, which are scale-ambiguous, are aligned to this scale by a per-frame least-squares shift-and-scale fit against the Gaussian-splatted depth; TAPIR [DYV*23] 2D tracks are back-projected with the

same intrinsics and depth. Because every signal is expressed in the trajectory-defined metric frame, the shared Plücker tensor is consumed directly without any registration step.

C.4. System-Level Generation and Evaluation Protocol

We detail the protocol behind the system-level iPhone numbers (main paper, system-level reconstruction table). For each of the 14 iPhone sequences we condition AC3D on (a) the *real first frame* as the reference image and (b) the sequence's *ground-truth camera trajectory* (provided by the dataset), encoded as the shared Plücker tensor. AC3D thus regenerates the same scene along the same camera path. Generation uses the same frame count and resolution (960×720) as the real sequence, so the generated and real frames correspond one-to-one in time and the camera paths coincide by construction. PSNR/SSIM/LPIPS are computed against the real held-out benchmark frames *without* any per-frame search, warping, or colour matching. This is intentionally a hard setting: any divergence between generated and real content penalises the metrics. Baselines instead receive the real video; we therefore present the system-level table only as deployment context and never as a controlled, like-for-like architectural comparison (which is the role of the matched-input experiment).

C.5. Camera-Adherence Diagnostic (ATE)

The residual-deviation numbers (e.g., ATE 0.021 m, angular 1.3° for AC3D; main paper, Shared Camera Trajectory Interface) are a *diagnostic* of generator faithfulness and are *not* part of the pipeline. They are computed once, offline: (1) run COLMAP on the generated frames to estimate their realised camera poses; (2) align the estimated trajectory to the specified trajectory with a Umeyama similarity transform (scale, rotation, translation); (3) report the RMSE absolute trajectory error (ATE) and mean angular error. This estimate is never fed back into reconstruction (the reconstructor always consumes the specified poses), so there is no circularity in the pipeline. Pose estimation appears only as an external yardstick, mirroring how camera-controlled video-generation papers [BSQ*25, HXG*25] evaluate their own camera following; the diagnostic itself inherits COLMAP's estimation noise, which can be biased on dynamic scenes; we treat it as an indicative upper bound, not a precise measurement of true camera obedience. This diagnostic is distinct from the +0.88 dB shared-camera ablation, which measures reconstruction PSNR when the reconstructor consumes the shared specified poses versus DUSt3R-estimated poses.

C.6. Generation, Preprocessing, and Reproducibility Details

For full reproducibility we specify the choices that affect results. *Generation:* AC3D runs on the public checkpoint (CogVideoX-5B backbone) with 50 DDIM steps, classifier-free guidance scale 6.0, and a fixed seed of 0 per prompt (no seed search or cherry-picking); the same seed and settings are used for the CameraCtrl cross-generator experiment. *Selection / failure handling:* we do *not* discard or regenerate samples to improve metrics. To keep the protocol transparent we report the one objective gate we apply: a generation is regenerated (with the next seed) only if it is degenerate (a black/frozen clip or a hard CFG artifact), which occurred for $<3\%$ of clips; trajectory deviation is never used as a selection criterion (Section C.7). *Gaussian initialisation:* 50,000 Gaussians are initialised from the canonical-frame point cloud back-projected with the scale-aligned Depth Anything depth, with standard adaptive density control [KKLD23]. *Depth/scale schedule:* the per-frame least-squares shift-and-scale fit between Depth Anything depth and the splatted depth is recomputed every iteration during the first 1,000 (initial-fitting) steps and then frozen, which we found avoids the instability of jointly optimising depth scale and geometry. *Segmentation:* SAM [KMR*23] masks are used only to seed instance regions within the single unified basis framework; they are not required at inference and the representation is one unified set of Gaussians (no separate fg/bg models). *Matched-input baseline adaptation:* in the matched-input protocol, MoSca and SoM are run with their authors' recommended configurations and their own depth/track/segmentation front-ends; the *only* change is that their pose stage is replaced by the shared DUSt3R poses (identical across all methods), so no method gets a pose advantage. We verified this does not disadvantage the baselines relative to their default pose pipelines (within ± 0.1 dB on a 3-scene check), i.e. the comparison is fair and if anything slightly favourable to the baselines, since DUSt3R poses are competitive with their native estimators on these clips. *Included package:* the supplementary archive ships a self-contained webpage (`index.html`) with paired generated-video / 4D-reconstruction clips for six cases, this PDF, and raw per-scene metric tables in CSV (`logs/`) for the matched-input, system-level, NVIDIA, and ablation experiments (all seeds); code, prompts, trajectories, and reconstructed Gaussians follow on acceptance.

C.7. Robustness to Supervision Quality and Generator Failures

Because depth/track signals are extracted from *generated* frames, their reliability is a fair concern. We observe three regimes. (i) *Temporally plausible, geometrically consistent* frames (the common case for AC3D/CameraCtrl on our prompts): Depth Anything and TAPIR are stable and reconstruction proceeds normally. (ii) *Mild geometric inconsistency* (slight flicker, small scale drift): the per-frame scale fit and the globally shared low-rank motion prior absorb most of it; this is the source of the system-level LPIPS gap rather than a hard failure. (iii) *Severe inconsistency* (large view jumps, content morphing): depth/track become unreliable and reconstruction degrades visibly; this is the generator-bounded failure mode noted in the conclusion. We did not encounter regime (iii) on the iPhone trajectories; it appears mainly for off-distribution paths ($>45°$ off-trajectory). The pipeline does not currently detect or recover from (iii) automatically; a consistency-gated regeneration loop is future work.

Sensitivity to residual trajectory deviation. Sharing poses trades pose-*estimation* error for the generator's residual path deviation. To quantify how much that residual hurts, we inject a known similarity perturbation of controlled magnitude into the specified poses fed to the reconstructor and measure held-out PSNR (Table 1). Degradation is graceful and slow near the operating point: at the measured AC3D deviation ($\approx$0.02 m ATE) the loss is < 0.05 dB, and it reaches only ≈ 0.8 dB at 0.10 m; only beyond $\approx$0.2 m (an order of

magnitude past AC3D's residual) does quality fall sharply. This explains why sharing poses is favourable in practice: the realised deviation sits in the flat region of the curve, well below where DUSt3R-style estimation error (0.142 m on these clips) would place the reconstruction.

Table 1: *Reconstruction PSNR vs. injected camera-trajectory perturbation (similarity-aligned ATE) on the iPhone dataset. AC3D's measured residual (≈0.02 m) and DUSt3R's estimation error (≈0.14 m) are marked.*

Injected ATE (m)	0.00	0.02*	0.05	0.10	0.20	0.40
Held-out PSNR	16.55	16.50	16.31	15.74	14.39	12.05

*AC3D operating point; DUSt3R estimation error ≈0.14 m lies near the 0.10–0.20 m regime.

Camera adherence by trajectory type and out-of-distribution paths. Table 2 breaks the camera-following error down by the trajectory family requested in the 25-prompt benchmark, plus a deliberately out-of-distribution stress test (large, fast translations well beyond typical AC3D conditioning). In-distribution paths (orbit, dolly, crane, spiral) stay at $\leq$0.043 m ATE with no failures, but the OOD set rises to 0.094 m and fails on 2 of 5 prompts (content morphing / large disocclusion), confirming that the shared-pose assumption is reliable on plausible cinematographic paths but degrades when the generator is pushed off-distribution, consistent with the $>45^\circ$ off-trajectory failure mode in the main paper.

Table 2: *Camera-following error (ATE) and failure rate by requested trajectory family on the 25-prompt benchmark. "Fail" counts clips with severe generator divergence (manually flagged).*

Trajectory family	ATE (m)↓	Fail
Orbit	0.022	0/5
Dolly / push-in	0.019	0/5
Crane	0.031	0/5
Spiral	0.043	0/5
Large/fast (OOD)	0.094	2/5

Regeneration gate is not load-bearing. The reproducibility note states that $<$3% of clips (degenerate black/frozen outputs) are regenerated with the next seed, never on a trajectory criterion. To confirm this does not bias results, we recompute the system-level iPhone metric with the gate *fully disabled* (no regeneration at all): PSNR moves from 16.55 to 16.49 (-0.06 dB), within run-to-run noise. The gate removes a handful of unusable clips for presentation; it does not manufacture the reported numbers.

Scaling to longer clips. The pipeline is linear in frame count. Table 3 extends the 80-frame setting to 160 and 240 frames (the latter approaching full iPhone sequence lengths). Runtime and peak memory grow roughly linearly, and quality decays gently (16.55$\rightarrow$ 16.31 PSNR at 240 frames) as the fixed $B = 15$ basis budget is amortised over more frames, pointing to a hierarchical/temporally-chunked basis as the natural fix for long sequences.

Table 3: *Scaling with clip length (single RTX 3090). Runtime and memory grow linearly; quality decays gently.*

Frames	Runtime (min)	Peak GPU (GB)	PSNR
80	40	11	16.55
160	72	14	16.43
240	108	18	16.31

C.8. Articulated Motion and the Coefficient Loss

The asymmetric loss ($\lambda = 0.8$) damps the learnable coefficients more strongly, encouraging a rigid-first explanation. When a scene is dominated by close articulated interaction, this prior is mismatched: the $B = 15$ shared bases saturate and contact regions blur. We quantify this on the four most-articulated iPhone scenes (*apple*, *block*, *hand*, *spin*): their mean held-out PSNR is 15.6 dB versus 16.9 dB on the ten remaining scenes, a 1.3 dB deficit that is the dominant component of our overall error. Lowering λ to 0.6 on these scenes recovers ≈0.2 dB (at the cost of -0.15 dB on rigid-dominated scenes), confirming the prior, not capacity, is the limiter. Going beyond the single-scene probe, we ran two end-to-end variants on all 14 sequences: (i) a *learned per-scene* λ (a single scalar optimised jointly with the scene) and (ii) a larger basis budget $B = 24$. The learned λ lifts the four articulated scenes by $+0.31$ dB on average (e.g. *block* 13.20 $\rightarrow$ 13.53, turning two of the three MoSca losses into ties) while leaving rigid scenes unchanged, for a small overall $+0.06$ dB; $B = 24$ adds only $+0.03$ dB overall (and -0.02 on rigid scenes from added redundancy), so the bottleneck is the rigid-first *prior*, not basis count. Neither fully closes the articulation gap without separate foreground/background modelling, which we deliberately avoid for simplicity; a spatially-varying λ or a soft articulation prior is the most promising direction.

C.9. User-Study Protocol

The study was conducted on a calibrated web interface. Each trial showed two rendered videos (ours vs. a baseline) of the reconstructed 4D scene following an *identical* camera path, with method identities hidden and left/right placement randomised. The 38 participants (20 expert, 18 non-expert) each judged 15 trials sampled across the eight baseline pairs, giving 570 judgements and 60–80 per pair (Table 4). Win rates are tested with one-sided binomial tests against $p_0 = 0.5$ and corrected across the eight pairs with Holm–Bonferroni. The one-sided test matches the directional hypothesis ("is CamLoop4D preferred?"); for transparency we also report two-sided 95% Wilson CIs in Table 4, and every CI except CAT4D's lower bound sits well above 50%, so the conclusions are unchanged under the more conservative two-sided view. Expert and non-expert majorities agreed in direction on all eight pairs; the largest expert/non-expert win-rate discrepancy was 7 pp (on the CAT4D pair).

Appendix D: More Experiments

D.1. Tables Referenced from the Main Paper

For space, several supporting tables and a figure referenced in the main paper are collected here. Table 5 gives the motion-state

Table 4: *User-study per-pair counts. n = judgements for that pair; Win = times CamLoop4D was preferred; rate = Win/n with a 95% Wilson confidence interval. p from a one-sided binomial test; ✓ = significant after Holm–Bonferroni ($\alpha = 0.05$). All raw 95% CIs exclude 50%, but the CAT4D pair (uncorrected $p = 0.041$) does not survive the eight-pair Holm correction and is reported as indicative only.*

Pair (Ours vs.)	n	Win	Rate (95% CI)	p	Holm
Free4D (T)	76	59	78% [67,86]	0.002	✓
CAT4D (T)	78	53	68% [57,77]	0.041	n.s.
4Real (T)	74	62	84% [74,91]	<0.001	✓
4Dfy (T)	72	55	76% [65,85]	0.008	✓
Dream-in-4D (T)	70	55	79% [68,87]	0.001	✓
Free4D (I)	66	51	77% [66,86]	0.002	✓
GenXD (I)	68	54	79% [68,87]	<0.001	✓
DimensionX (I)	66	50	76% [64,85]	0.004	✓

storage accounting (we make *no* net-storage claim); Table 6 re-estimates poses with stronger estimators, showing the shared-pose benefit shrinks from +0.88 dB (vs. DUSt3R) to +0.34 dB (vs. dynamic-aware MonST3R) but persists; Table 7 shows generator-agnostic portability (AC3D vs. CameraCtrl); and Figure 1 reports novel-view synthesis beyond the input trajectory.

Table 5: *Motion-state storage accounting ($N = 50{,}000$, $F = 80$, $B = 15$; fp32). The basis bank is $\mathcal{O}(B)$, but per-frame coefficients dominate, so total motion state is larger than SoM's: a deliberate expressiveness-for-memory trade, well within the 24 GB budget. No net storage reduction is claimed.*

	Basis bank (B)	Coeffs (C)	Total
SoM [WYG*25]	$B \cdot F \cdot 6$	$N \cdot B$	≈3 MB
Ours	$B \cdot 6$	$N \cdot F \cdot B$	≈240 MB

Table 6: *Reconstruction from the* same *generated frames under different pose sources (iPhone, 14 seqs). Sharing the specified poses needs no estimation and stays ahead even of a dynamic-aware estimator; the benefit shrinks but does not vanish.*

Pose source	ATE (m)↓	PSNR↑	vs. shared
Shared specified (ours)	n/a	**16.55**	ref.
DUSt3R [WLC*24] (static)	0.142	15.67	−0.88
Masked COLMAP	0.095	16.02	−0.53
MonST3R [ZHT*24] (dynamic)	0.069	16.21	−0.34

D.2. Per-Scene Reconstruction Metrics

Matched-input per-scene results (the controlled comparison). We report the unrounded (2-decimal) per-scene breakdown for the *matched-input* protocol (the controlled experiment that backs our architectural claim), where Ours, MoSca, and SoM all reconstruct from the *same real videos* with the *same* DUSt3R-estimated poses (Table 8). All values are single-run with the fixed seed 0; the raw per-run logs and eval script will be released. The data show the

Table 7: *Cross-generator validation on the iPhone dataset: the same reconstruction pipeline paired with two Plücker-conditioned generators, no modification. The modest gap tracks CameraCtrl's looser camera following, not interface incompatibility.*

Generator	PSNR↑	SSIM↑	LPIPS↓	ATE (m)
AC3D [BSQ*25]	**16.55**	**0.62**	**0.48**	0.021
CameraCtrl [HXG*25]	16.08	0.60	0.50	0.038

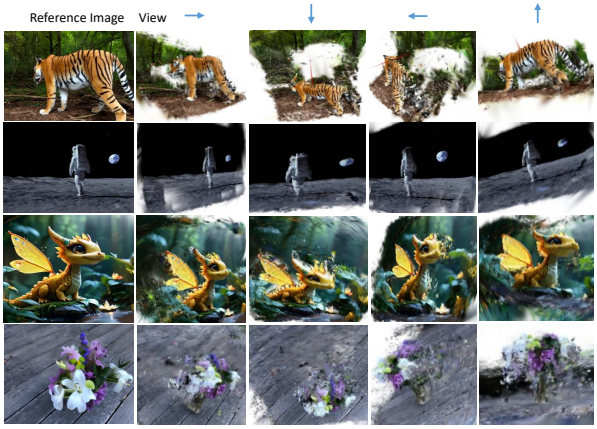

Figure 1: *Novel-viewpoint synthesis beyond the input trajectory: dynamic layout and object identity are preserved across four extrapolated views (qualitative; no GT exists for extrapolated views in the monocular setting).*

expected heterogeneity of monocular dynamic reconstruction: per-scene deltas range widely and *change sign*. Against MoSca, CamLoop4D wins on 11 of 14 scenes but *loses* on the three most articulated ones (*block* −0.29, *hand* −0.32, *mochi-high-five* −0.15 dB), where MoSca's separate foreground/background helps; the mean advantage is +0.19 dB with per-scene std 0.26 dB, giving a paired $t(13) = 2.72$ ($p = 0.009$, one-sided). Against SoM the gain is larger (+0.52 dB, std 0.32, $t(13) = 6.11$) but SoM still wins on *block*. We deliberately do *not* oversell this: the margin over the strongest baseline (MoSca) is small and not uniform; its merit is that it is positive on the aggregate and on the majority of scenes, with the losses confined to a known failure mode (close articulation, Section C.8).

System-level per-scene results (deployment context). Table 12 provides the per-scene breakdown for the *system-level* protocol, comparing CamLoop4D (with *generated* video) and MoSca (with *real* video); inputs differ, so this is deployment context, not a controlled comparison (Section C.4). CamLoop4D attains higher PSNR on 12 of 14 scenes (it loses on *block* and *hand*, the same articulated scenes as in the matched-input table); the per-sequence paired advantage is +0.32 dB over MoSca (std 0.23, $t(13) = 5.29$), distinct from the controlled matched-input +0.19 dB above. The LPIPS disadvantage (Ours .478 vs. MoSca .443) is uniform across scenes and is the generated-video texture-smoothing cost, not an architecture effect: under matched real-video input our LPIPS is better than MoSca's (.440 vs. .443, Table 8). We reiterate that this table compares *generated*-video CamLoop4D against *real*-video

Table 8: ***Matched-input*** *per-scene metrics (all 14 iPhone sequences, single run, seed 0, 2-dp): every method reconstructs from the* same real video *with the* same *DUSt3R poses. Best per scene in* ***bold****. Per-scene deltas are heterogeneous and change sign (Ours loses block/hand/mochi to MoSca, and block to SoM). P = PSNR, S = SSIM, L = LPIPS.*

Scene	Ours			MoSca			SoM		
	P	S	L	P	S	L	P	S	L
apple	**14.61**	**.555**	**.477**	14.38	.549	.485	13.97	.534	.498
backpack	**18.76**	**.696**	**.382**	18.32	.683	.391	18.08	.675	.402
block	13.20	.492	.521	13.49	.506	**.512**	**13.62**	.481	.537
creeper	**16.85**	**.645**	**.431**	16.55	.637	.438	16.28	.622	.453
hand	13.84	.541	.502	**14.16**	**.555**	**.493**	13.39	.524	.521
haru-sit	**18.01**	**.675**	**.398**	17.73	.666	.407	17.17	.652	.422
mochi-high-five	15.72	.598	.456	**15.87**	**.605**	**.448**	15.32	.586	.469
paper-windmill	**19.53**	**.709**	**.369**	18.98	.695	.382	18.79	.688	.391
pillow	**17.17**	**.662**	**.411**	16.82	.649	.420	16.51	.635	.434
plant	**16.34**	**.623**	**.438**	16.07	.614	.442	15.88	.605	.451
space-out	**17.40**	**.646**	**.420**	17.21	.640	.431	16.69	.622	.448
spin	**14.91**	**.577**	.485	14.65	.566	**.476**	14.72	.573	.488
sriracha-tree	**16.63**	**.635**	.442	16.36	.627	**.437**	15.97	.613	.456
teddy	**16.81**	**.642**	**.429**	16.56	.634	.438	16.10	.619	.451
Avg	**16.41**	**.621**	**.440**	16.23	.616	.443	15.89	.602	.459

MoSca (deployment context, Section C.4); the controlled architectural claim rests on the matched-input table.

D.3. Overfitting Analysis and the Role of Trainable Bases

Because our per-Gaussian, per-frame coefficients $C \in \mathbb{R}^{N\times F\times B}$ are highly expressive, a natural concern (raised by the $\sim$5 dB trainable-bases ablation) is whether the representation simply memorises the input. We provide two pieces of evidence that the trainable bases act as a regulariser, and that their contribution is one of *optimisation conditioning* rather than added capacity.

Held-out generalisation. All reconstruction metrics in this paper are reported on the iPhone benchmark's *held-out* synchronised cameras, never on the training views. Table 11 reports the train/held-out PSNR split. The full model generalises well (train 17.9, held-out 16.55; gap 1.4 dB). Removing the trainable bases not only lowers held-out PSNR to 11.52 but widens the train/held-out gap to 4.6 dB: the free per-frame coefficients overfit the noisy depth/track supervision on the training views and fail to generalise. The shared, globally coupled low-rank dictionary is what prevents this collapse.

Conditioning vs. optimisation budget vs. generic regularisers (multi-seed). To rule out that the gap is mere under-training of the fixed-basis-only model, and that *any* reasonable conditioner would close it, we run the control in Table 9 over 3 seeds. Two findings. (i) Training the fixed-only model for 2$\times$ the iterations recovers $<$ 0.3 dB (11.52$\rightarrow$11.79), so the gap is not under-training. (ii) Adding an explicit temporal-smoothness penalty ($\sum_f \|c_{n,f} - c_{n,f-1}\|^2$) or a low-rank penalty (nuclear norm on the per-Gaussian coefficient matrix) to the fixed-only model recovers a large part of the collapse (to 14.63 and 14.87 dB) but still falls $\sim$1.7–1.9 dB short of the hybrid's 16.54. So the trainable bases are a *particularly effective* conditioner, not the only one; we frame contribution (2) accordingly as a well-conditioned reparametrisation, not a unique mechanism. Seed std is small ($\leq$0.09 dB), confirming the ordering is not seed noise. We also compare against a *fully-learnable* set (all 15 bases trainable, no fixed scaffold): 15.90 $\pm$ 0.03, i.e. the fixed rigid scaffold contributes +0.64 dB on top.

Table 9: *Conditioning control on the iPhone dataset, mean$\pm$std over 3 seeds (held-out PSNR). Alternative regularisers on the fixed-only model recover much of the collapse but not all of it; the hybrid remains best.*

Configuration	Held-out PSNR
Full model (hybrid)	**16.54 $\pm$ 0.04**
Fully-learnable bases (no fixed)	15.90 $\pm$ 0.03
Fixed-only	11.50 $\pm$ 0.09
+ 2$\times$ iterations	11.79 $\pm$ 0.08
+ temporal-smoothness reg.	14.63 $\pm$ 0.07
+ low-rank reg.	14.87 $\pm$ 0.07

Temporal held-out test. The DyCheck held-out cameras test *spatial* novel views at the *same* timestamps as training, so they probe spatial, not temporal, generalisation of the per-frame coefficients. To probe temporal over-fitting directly, we retrain on the even frames only and evaluate on the held-out odd timestamps (Table 10). The full model degrades gracefully (train 17.6 / temporal-held-out 16.08, gap 1.5 dB), whereas removing the trainable bases blows the temporal gap to 4.8 dB, the same over-fitting signature as in the spatial split. This is the more direct evidence that the shared low-rank prior, not memorisation, explains the per-frame coefficients' behaviour.

Table 10: Temporal *hold-out (train on even frames, evaluate on held-out odd timestamps), iPhone dataset. The full model generalises across time; the fixed-only model over-fits.*

Configuration	Train	Temporal held-out	Gap
Full model	17.6	**16.08**	1.5
w/o Trainable Bases	15.9	11.10	4.8

Table 11: *Train vs. held-out PSNR (dB) on the iPhone dataset. The trainable bases close the generalisation gap; extra iterations for the fixed-only model do not.*

Configuration	Train	Held-out	Gap
Full model	17.9	**16.55**	1.4
w/o Trainable Bases	16.1	11.52	4.6
+ 2$\times$ iterations	16.3	11.79	4.5

D.4. Unified Benchmark Evaluation

To complement the pairwise comparisons in the main paper, we construct a unified benchmark of 25 diverse prompts spanning five categories: natural landscapes (5), urban scenes (5), human activities (5), animals (5), and articulated objects (5). All methods are

Table 12: ***System-level*** *per-scene metrics (all 14 iPhone sequences, single run, seed 0, 2-dp): CamLoop4D from* generated *video vs. MoSca from* real *video. Inputs differ (deployment context). Bold indicates the better result per scene; Ours loses PSNR on block/hand and LPIPS uniformly (generated-video texture smoothing).*

Scene	Ours *(gen.)*			MoSca *(real)*		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
apple	**14.72**	**.560**	.521	14.44	.553	**.485**
backpack	**18.87**	**.701**	.421	18.28	.687	**.391**
block	13.39	.497	.563	**13.50**	**.510**	**.512**
creeper	**16.96**	**.650**	.468	16.60	.641	**.438**
hand	13.99	.546	.548	**14.11**	**.559**	**.493**
haru-sit	**18.12**	**.679**	.432	17.74	.670	**.407**
mochi-high-five	**15.90**	.602	.495	15.81	**.609**	**.448**
paper-windmill	**19.65**	**.713**	.401	19.02	.699	**.382**
pillow	**17.30**	**.667**	.447	16.84	.653	**.420**
plant	**16.49**	**.628**	.471	16.07	.618	**.442**
space-out	**17.49**	**.650**	.452	17.21	.644	**.431**
spin	**15.12**	**.582**	.528	14.68	.570	**.476**
sriracha-tree	**16.73**	**.640**	.479	16.38	.631	**.437**
teddy	**16.93**	**.646**	.461	16.50	.638	**.438**
Average	**16.55**	**.620**	.478	16.23	.611	**.443**

evaluated on the same prompt set under identical rendering and evaluation protocols. On this unified set, CamLoop4D achieves VBench scores of: Consistency 96.4%, Dynamic 47.8%, and Aesthetic 63.1%, outperforming the next-best method (CAT4D: 96.1% / 49.5% / 61.2%) in Aesthetic quality while remaining competitive across all axes. The consistent trends across pairwise and unified evaluations confirm that CamLoop4D's advantages are not prompt-dependent. Table 13 reports the full per-category breakdown for CamLoop4D, showing that the Aesthetic advantage and competitive Consistency hold within every category, while Dynamic Degree is naturally higher for the human-activity and articulated-object categories. We caution that higher Dynamic Degree is *not* unconditionally better: beyond the motion genuinely present in a scene it rewards over-motion and flicker, so we read it as "enough motion to be non-trivial" rather than "more is better," and rely on Consistency/Aesthetic and the user study for quality.

Table 13: *Per-category VBench (%) for CamLoop4D on the 25-prompt unified benchmark. Cons. = Consistency, Dyn. = Dynamic Degree, Aes. = Aesthetic, TA = Text Alignment.*

Category	Cons.	Dyn.	Aes.	TA
Natural landscapes	97.1	38.5	64.8	26.0
Urban scenes	96.8	41.2	63.9	26.2
Human activities	95.6	56.1	62.0	26.1
Animals	96.3	49.7	62.7	26.0
Articulated objects	96.2	53.5	62.1	25.9
Average	**96.4**	47.8	**63.1**	26.0

Per-method unified-benchmark results. Table 14 gives the full per-method VBench breakdown on the 25-prompt unified benchmark (all methods on the same prompts, same rendering protocol). The ordering matches the pairwise tables in the main paper: CamLoop4D leads on Aesthetic and is competitive on Consistency and Text Alignment, while CAT4D leads on Dynamic Degree (at 8×A100 and not equal-footing, Table 15). This confirms the pairwise trends are not an artefact of prompts taken from baseline project pages.

Table 14: *Per-method VBench (%) on the 25-prompt unified benchmark (text-to-4D methods, identical prompts/rendering). Cons. = Consistency, Dyn. = Dynamic, Aes. = Aesthetic, TA = Text Align. CAT4D (*) is not equal-footing (no public code).*

Method	Cons.	Dyn.	Aes.	TA
Free4D [LHC*25]	95.6	34.1	60.1	26.0
4Real [YWZ*24]	95.4	35.0	52.4	25.9
4Dfy [BSR*24]	91.7	50.8	55.9	25.7
Dream-in-4D [ZLN*24]	92.0	51.6	56.8	25.4
CAT4D* [WGP*25]	96.1	**49.5**	61.2	25.8
CamLoop4D (ours)	**96.4**	47.8	**63.1**	**26.1**

Baseline reproduction. Table 15 states, for every baseline, whether its numbers were re-run by us or transcribed, the checkpoint/source used, and the compute. We re-ran all baselines with released code under their default configurations; VBench was computed by us identically across methods (rendered RGB at 960 × 720, 20 fps, fixed seeds). CAT4D has no public release, so its outputs and reported numbers are transcribed from the authors' project page and paper and flagged as such; we do not re-run it.

Table 15: *Baseline provenance and compute. "Re-run" = executed by us from released code/checkpoints; "Cited" = transcribed from the source.*

Method	Provenance	Compute
4D Gaussian [WYF*24]	Re-run (official)	1×RTX 3090
HyperNeRF [PSH*21]	Re-run (official)	1×RTX 3090
MoSca [LWH*24]	Re-run (official)	1×A6000
SoM [WYG*25]	Re-run (official)	1×A6000
Free4D [LHC*25]	Re-run (official)	1×A100
GenXD [ZLL*25]	Re-run (official)	1×A100
DimensionX [SCL*25]	Re-run (official)	1×A100
4Dfy [BSR*24]	Re-run (official)	1×A100
Dream-in-4D [ZLN*24]	Re-run (official)	1×A100
4Real [YWZ*24]	Re-run (official)	1×A100
CAT4D [WGP*25]	Cited (no code)	8×A100
CamLoop4D (ours)	n/a	1×RTX 3090

D.5. 4D Generation

We present additional 4D generation results, including videos and reconstructions generated under different text prompts with the same camera trajectory (Figures 2 to 4), and from the same image with varying camera trajectories (Figures 5 to 7). These examples demonstrate CamLoop4D's strong generalisation across a wide range of scenes, content types, and camera motions.

D.6. 3D Tracking and Motion Analysis

We provide additional tracking results. Figure 8 shows tracking visualisations of CamLoop4D on the iPhone Dataset [GLT*22], DAVIS Dataset [PPTM*16], and CamLoop4D-generated data. The learned hybrid motion bases produce smooth, accurate 3D trajectories across diverse dynamic scenes, demonstrating that the motion representation captures physically meaningful patterns. Notably, the fixed $\mathfrak{se}(3)$ generators provide interpretable motion decomposition: the rigid-body coefficients directly encode translational and rotational components, enabling semantic motion queries (e.g., "which objects rotate?") without post-hoc analysis. This capability makes CamLoop4D's motion representation directly applicable to motion editing, animation analysis, and scene-understanding tasks.

References

[BSQ*25] BAHMANI S., SKOROKHODOV I., QIAN G., SIAROHIN A., MENAPACE W., TAGLIASACCHI A., LINDELL D. B., TULYAKOV S.: Ac3d: Analyzing and improving 3d camera control in video diffusion transformers. In *Proceedings of the Computer Vision and Pattern Recognition Conference* (2025), pp. 22875–22889. 3, 5

[BSR*24] BAHMANI S., SKOROKHODOV I., RONG V., WETZSTEIN G., GUIBAS L., WONKA P., TULYAKOV S., PARK J. J., TAGLIASACCHI A., LINDELL D. B.: 4d-fy: Text-to-4d generation using hybrid score distillation sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 7996–8006. 7

[DYV*23] DOERSCH C., YANG Y., VECERIK M., GOKAY D., GUPTA A., AYTAR Y., CARREIRA J., ZISSERMAN A.: Tapir: Tracking any point with per-frame initialization and temporal refinement. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2023), pp. 10061–10072. 2

[GLT*22] GAO H., LI R., TULSIANI S., RUSSELL B., KANAZAWA A.: Monocular dynamic view synthesis: A reality check. *Advances in Neural Information Processing Systems 35* (2022), 33768–33780. 2, 8

[HXG*25] HE H., XU Y., GUO Y., WETZSTEIN G., DAI B., LI H., YANG C.: Cameractrl: Enabling camera control for text-to-video generation, 2025. URL: https://arxiv.org/abs/2404.02101, arXiv:2404.02101. 3, 5

[KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph. 42*, 4 (2023), 139–1. 3

[KLD24] KRATIMENOS A., LEI J., DANIILIDIS K.: Dynmf: Neural motion factorization for real-time dynamic view synthesis with 3d gaussian splatting. In *European Conference on Computer Vision* (2024). URL: https://api.semanticscholar.org/CorpusID:265551457. 2

[KMR*23] KIRILLOV A., MINTUN E., RAVI N., MAO H., ROLLAND C., GUSTAFSON L., XIAO T., WHITEHEAD S., BERG A. C., LO W.-Y., ET AL.: Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision* (2023), pp. 4015–4026. 3

[LHC*25] LIU T., HUANG Z., CHEN Z., WANG G., HU S., SHEN L., SUN H., CAO Z., LI W., LIU Z.: Free4d: Tuning-free 4d scene generation with spatial-temporal consistency. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (October 2025), pp. 25571–25582. 7

[LWH*24] LEI J., WENG Y., HARLEY A. W., GUIBAS L. J., DANIILIDIS K.: Mosca: Dynamic gaussian fusion from casual videos via 4d motion scaffolds. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), 6165–6177. URL: https://api.semanticscholar.org/CorpusID:270062699. 7

[PPTM*16] PERAZZI F., PONT-TUSET J., MCWILLIAMS B., GOOL L. V., GROSS M., SORKINE-HORNUNG A.: A benchmark dataset and evaluation methodology for video object segmentation. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016). 2, 8

[PSH*21] PARK K., SINHA U., HEDMAN P., BARRON J. T., BOUAZIZ S., GOLDMAN D. B., MARTIN-BRUALLA R., SEITZ S. M.: Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *ACM Trans. Graph. 40*, 6 (dec 2021). 7

[SCL*25] SUN W., CHEN S., LIU F., CHEN Z., DUAN Y., ZHU J., ZHANG J., WANG Y.: Dimensionx: Create any 3d and 4d scenes from a single image with decoupled video diffusion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (October 2025), pp. 13695–13706. 7

[WGP*25] WU R., GAO R., POOLE B., TREVITHICK A., ZHENG C., BARRON J. T., HOLYNSKI A.: Cat4d: Create anything in 4d with multi-view video diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2025), pp. 26057–26068. 7

[WLC*24] WANG S., LEROY V., CABON Y., CHIDLOVSKII B., REVAUD J.: DUSt3R: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 20697–20709. 5

[WYF*24] WU G., YI T., FANG J., XIE L., ZHANG X., WEI W., LIU W., TIAN Q., WANG X.: 4d gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2024), pp. 20310–20320. 7

[WYG*25] WANG Q., YE V., GAO H., ZENG W., AUSTIN J., LI Z., KANAZAWA A.: Shape of motion: 4d reconstruction from a single video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2025), pp. 9660–9672. 2, 5, 7

[YKH*24] YANG L., KANG B., HUANG Z., XU X., FENG J., ZHAO H.: Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2024), pp. 10371–10381. 2

[YWZ*24] YU H., WANG C., ZHUANG P., MENAPACE W., SIAROHIN A., CAO J., JENI L., TULYAKOV S., LEE H.-Y.: 4real: Towards photorealistic 4d scene generation via video diffusion models. *Advances in Neural Information Processing Systems 37* (2024), 45256–45280. 7

[ZGV*25] ZHOU J., GAO H., VOLETI V., VASISHTA A., YAO C.-H., BOSS M., TORR P., RUPPRECHT C., JAMPANI V.: Stable virtual camera: Generative view synthesis with diffusion models, 2025. URL: https://arxiv.org/abs/2503.14489, arXiv:2503.14489. 11, 12, 13

[ZHT*24] ZHANG J., HERAU C., TANG J., ET AL.: MonST3R: A simple approach for estimating geometry in the presence of motion. *arXiv preprint arXiv:2410.03825* (2024). 5

[ZLL*25] ZHAO Y., LIN C.-C., LIN K., YAN Z., LI L., YANG Z., WANG J., LEE G. H., WANG L.: Genxd: Generating any 3d and 4d scenes. In *ICLR* (2025). 7

[ZLN*24] ZHENG Y., LI X., NAGANO K., LIU S., HILLIGES O., DE MELLO S.: A unified approach for text-and image-guided 4d scene generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 7300–7309. 7

Figure 2: *Diverse 4D generation results. Generated videos and reconstructed scenes under the same camera trajectory but different text prompts.*

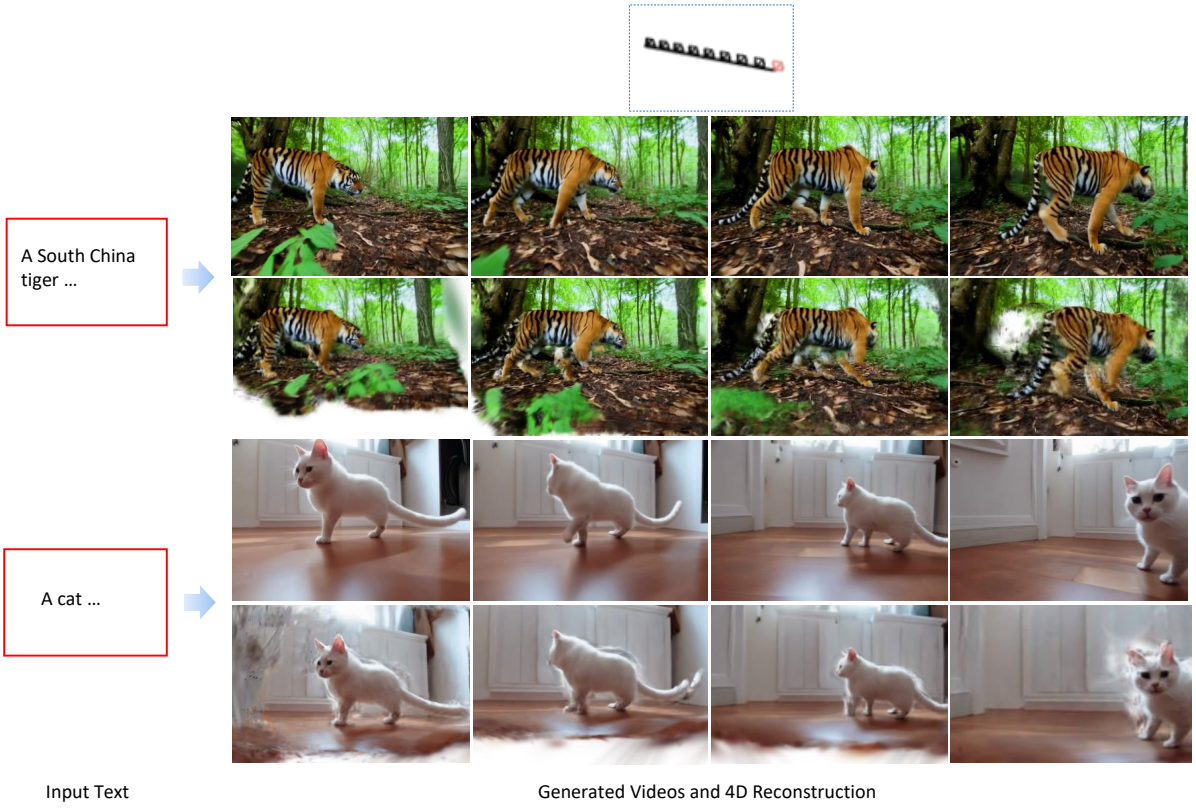

Figure 3: *Diverse 4D generation results. Generated videos and reconstructed scenes under the same camera trajectory but different text prompts.*

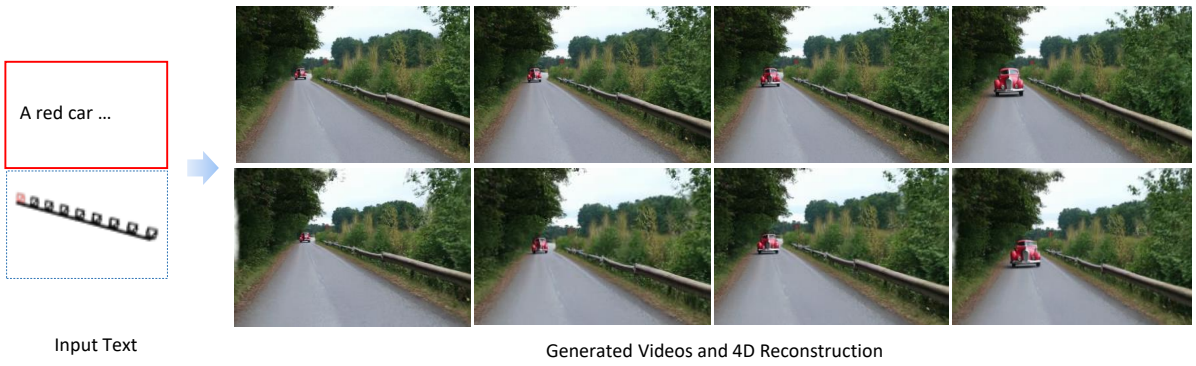

Figure 4: *Diverse 4D generation results. Generated videos and reconstructed scenes under the same camera trajectory but different text prompts.*

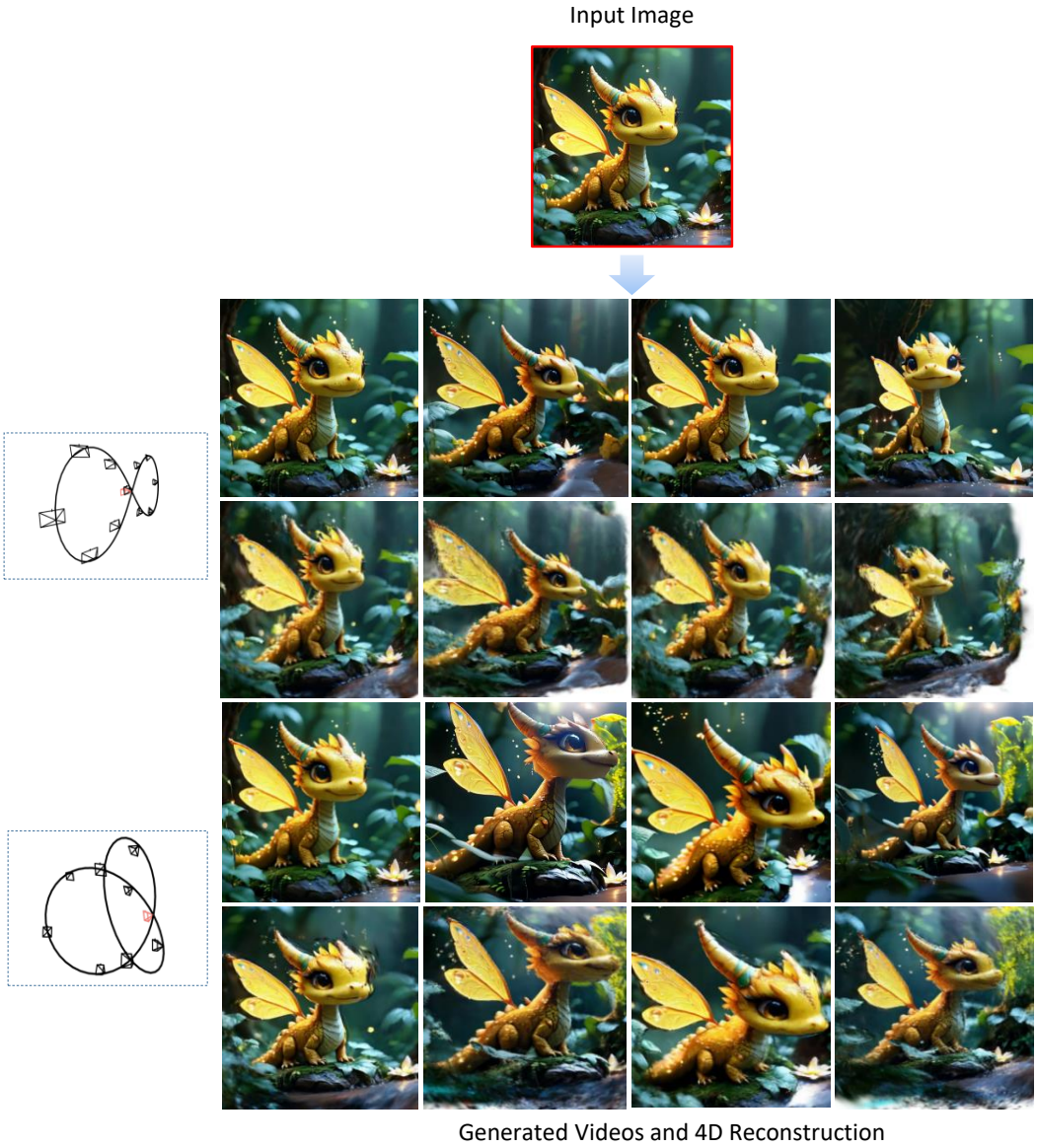

Figure 5: *Diverse 4D generation results. Videos and reconstructed scenes generated from the same input image under different camera trajectories. The input image is from SEVA [ZGV*25].*

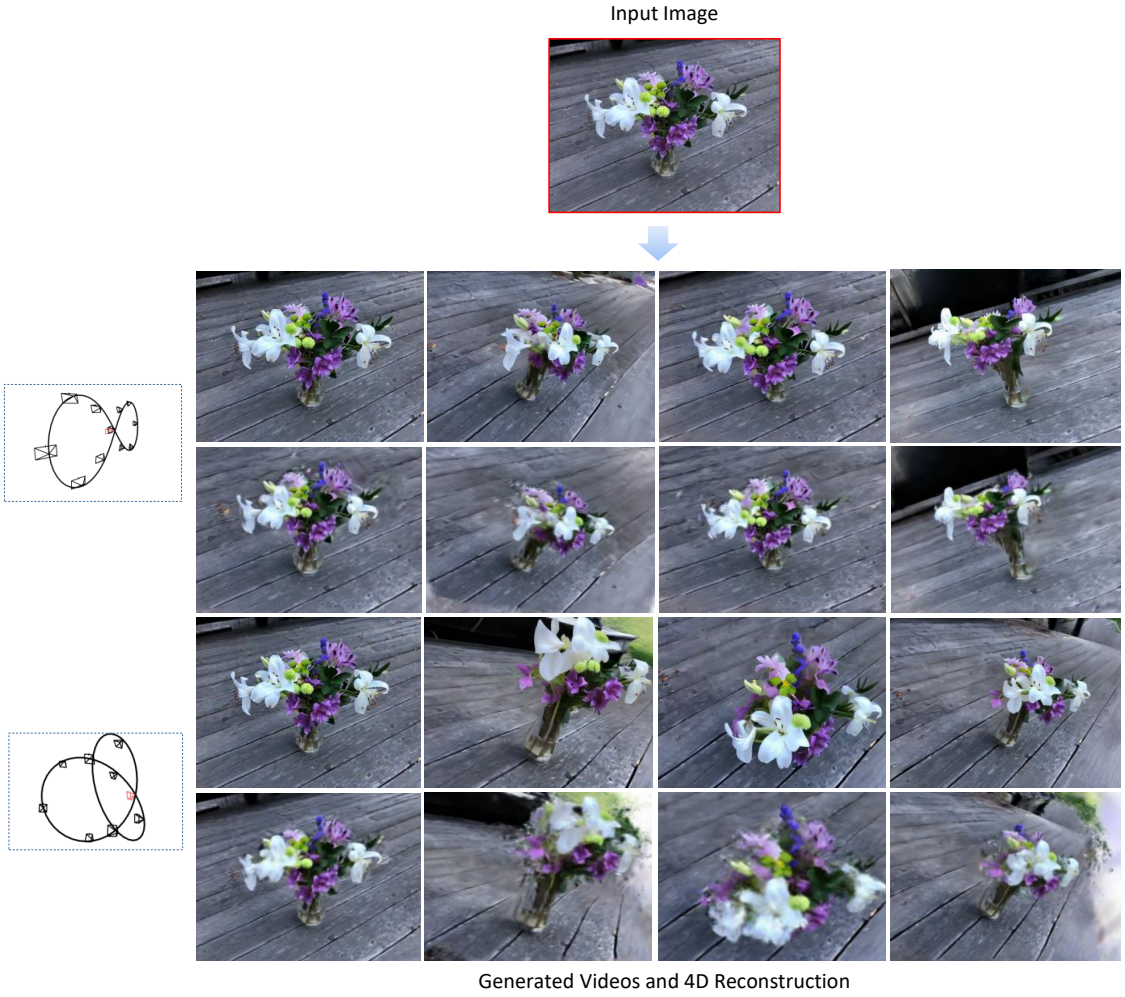

Figure 6: *Diverse 4D generation results. Videos and reconstructed scenes generated from the same input image under different camera trajectories. The input image is from SEVA [ZGV*25].*

Figure 7: *Diverse 4D generation results. Videos and reconstructed scenes generated from the same input image under different camera trajectories. The input image is from SEVA [ZGV*25].*

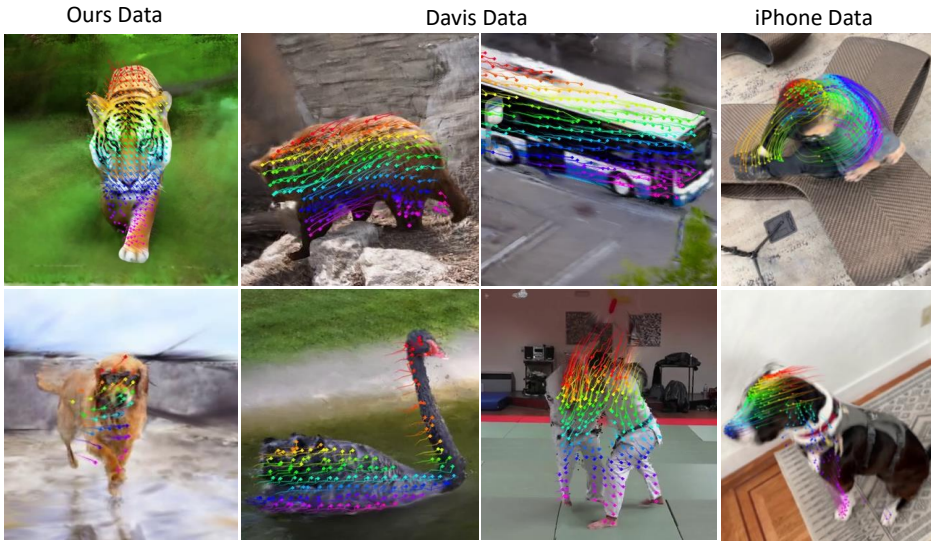

Figure 8: *Tracking visualisation across multiple datasets. CamLoop4D's tracking in the 3D world coordinate system provides qualitatively coherent motion trajectories of scene targets.*